# 3. Black Box Testing

## Test Cases

| Test Case ID | API / Scenario | Input | Expected Output | Justification for Inclusion |
|---|---|---|---|---|
| BB-01 | Missing roll header | `GET /admin/users` without `X-Roll-Number` | HTTP `401`, `error` in payload | Validates mandatory global authentication header. |
| BB-02 | Invalid roll header type | `GET /admin/users`, `X-Roll-Number: abc` | HTTP `400`, invalid integer error | Ensures strict roll-number parsing. |
| BB-03 | Missing user header | `GET /profile` without `X-User-ID` | HTTP `400`, missing user header error | Confirms user-scoped endpoint requires user identity. |
| BB-04 | Invalid user id (abc) | `GET /profile`, `X-User-ID: abc` | HTTP `400`, error string | Covers non-numeric ID input. |
| BB-05 | Invalid user id (-1) | `GET /profile`, `X-User-ID: -1` | HTTP `400`, error string | Covers negative ID boundary. |
| BB-06 | Invalid user id (0) | `GET /profile`, `X-User-ID: 0` | HTTP `400`, error string | Covers zero ID boundary. |
| BB-07 | Invalid user id (99999999) | `GET /profile`, large unknown user | HTTP `400` or `404`, error string | Covers validation/order variance with unknown ID. |
| BB-08 | Profile happy path | `GET /profile` for user `1` | HTTP `200`, expected profile keys | Verifies response format defined in API doc on valid request. |
| BB-09 | Profile name min boundary fail | `PUT /profile` with `name="A"` | HTTP `400`, name length error | Validates lower bound on name. |
| BB-10 | Profile name max boundary fail | `PUT /profile` with 51-char name | HTTP `400`, name length error | Validates upper bound on name. |
| BB-11 | Profile phone length fail | `PUT /profile` with short phone | HTTP `400`, phone length error | Validates exact phone format rule. |
| BB-12 | Profile missing field variant 1 | `PUT /profile` body `{ "name": "Valid User" }` | HTTP `400`, error string | Ensures `phone` is required. |

| Test Case ID | API / Scenario | Input | Expected Output | Justification for Inclusion |
|---|---|---|---|---|
| BB-13 | Profile missing field variant 2 | `PUT /profile` body `{ "phone": "9876543210" }` | HTTP `400`, error string | Ensures `name` is required. |
| BB-14 | Profile missing field variant 3 | `PUT /profile` body `{}` | HTTP `400`, error string | Covers empty payload boundary. |
| BB-15 | Profile wrong type variant 1 | `PUT /profile` with numeric `name` | HTTP `400`, error string | Validates type enforcement for `name`. |
| BB-16 | Profile wrong type variant 2 | `PUT /profile` with numeric `phone` | HTTP `400`, error string | Validates type enforcement for `phone`. |
| BB-17 | Product list filters and sort | `GET /products` with category/search/sort params | HTTP `200`, filtered + sorted + active-only | Validates storefront listing behavior. |
| BB-18 | Product lookup not found | `GET /products/999999` | HTTP `404`, `Product not found` | Verifies missing-resource behavior. |
| BB-19 | Product price consistency | Compare admin vs public product `123` price | HTTP `200`, exact price match expected | Detects price transformation defects. |
| BB-20 | Address label enum fail | `POST /addresses` with `label="BAD"` | HTTP `400`, label validation error | Validates enum constraints. |
| BB-21 | Address boundary variant 1 | `POST /addresses` with 4-char street | HTTP `400`, error string | Validates street minimum length. |
| BB-22 | Address boundary variant 2 | `POST /addresses` with 101-char street | HTTP `400`, error string | Validates street maximum length. |
| BB-23 | Address boundary variant 3 | `POST /addresses` with 1-char city | HTTP `400`, error string | Validates city minimum length. |
| BB-24 | Address boundary variant 4 | `POST /addresses` with 51-char city | HTTP `400`, error string | Validates city maximum length. |
| BB-25 | Address boundary variant 5 | `POST /addresses` with 5-digit pincode | HTTP `400`, error string | Validates pincode minimum length. |
| BB-26 | Address boundary variant 6 | `POST /addresses` with 7-digit pincode | HTTP `400`, error string | Validates pincode maximum length. |

| Test Case ID | API / Scenario | Input | Expected Output | Justification for Inclusion |
|---|---|---|---|---|
| BB-27 | Address missing field variant 1 | `POST /addresses` missing `label` | HTTP `400`, error string | Verifies required `label`. |
| BB-28 | Address missing field variant 2 | `POST /addresses` missing `street` | HTTP `400`, error string | Verifies required `street`. |
| BB-29 | Address missing field variant 3 | `POST /addresses` missing `city` | HTTP `400`, error string | Verifies required `city`. |
| BB-30 | Address missing field variant 4 | `POST /addresses` missing `pincode` | HTTP `400`, error string | Verifies required `pincode`. |
| BB-31 | Address wrong type variant 1 | `POST /addresses` numeric `label` | HTTP `400`, error string | Enforces `label` type. |
| BB-32 | Address wrong type variant 2 | `POST /addresses` numeric `street` | HTTP `400`, error string | Enforces `street` type. |
| BB-33 | Address wrong type variant 3 | `POST /addresses` numeric `pincode` | HTTP `400`, error string | Enforces `pincode` type. |
| BB-34 | Address wrong type variant 4 | `POST /addresses` string `is_default` | HTTP `400`, error string | Enforces boolean type. |
| BB-35 | Address update reflects mutation | Create then update address street/default | Expected HTTP `200` and updated fields in response | Verifies update response consistency. |
| BB-36 | Cart zero quantity fail | `POST /cart/add` with `quantity=0` | Expected HTTP `400`, error payload | Validates zero quantity rejection. |
| BB-37 | Cart negative quantity fail | `POST /cart/add` with `quantity=-1` | HTTP `400`, error payload | Validates negative quantity rejection. |
| BB-38 | Cart missing field variant 1 | `POST /cart/add` body `{ "quantity": 1 }` | HTTP `400/404`, error payload | Verifies required `product_id`. |
| BB-39 | Cart missing field variant 2 | `POST /cart/add` body `{}` | HTTP `400/404`, error payload | Verifies empty payload rejection. |
| BB-40 | Cart missing quantity explicit | `POST /cart/add` body `{ "product_id": 1 }` | HTTP `400/404`, error payload | Verifies required `quantity`. |

| Test Case ID | API / Scenario | Input | Expected Output | Justification for Inclusion |
|---|---|---|---|---|
| BB-41 | Cart wrong type variant 1 | `POST /cart/add` with `product_id` as string | HTTP `400`, error payload | Enforces numeric `product_id`. |
| BB-42 | Cart wrong type variant 2 | `POST /cart/add` with `quantity` as string | HTTP `400`, error payload | Enforces numeric `quantity`. |
| BB-43 | Cart missing product ID not found | `POST /cart/add` with `product_id=999999` | HTTP `404`, error payload | Validates product existence check. |
| BB-44 | Cart subtotal/total arithmetic | Add qty `2`, verify subtotal and total math | Expected subtotal/total by unit price | Validates billing arithmetic correctness. |
| BB-45 | Expired coupon rejection | `POST /coupon/apply` with `EXPIRED100` | HTTP `400`, `Coupon expired` | Validates coupon expiry rule. |
| BB-46 | Coupon minimum value enforcement | Apply `FIRSTORDER` below threshold | HTTP `400`, `Cart value below minimum` | Validates coupon minimum cart rule. |
| BB-47 | FIRSTORDER percent coupon max-cap check | Add high-value cart, apply `FIRSTORDER` | HTTP `200`; expected `discount=min(total×15%,150)` | Verifies max discount cap enforcement for percent coupon. |
| BB-48 | PERCENT20 coupon max-cap check | Add high-value cart, apply `PERCENT20` | HTTP `200`; expected `discount=min(total×20%,200)` | Validates cap logic for another percent coupon rule. |
| BB-49 | Checkout rejects empty cart | `POST /checkout` with empty cart | HTTP `400`, `Cart is empty` | Ensures checkout precondition on cart non-emptiness. |
| BB-50 | Checkout rejects missing payment method | `POST /checkout` with `{}` | HTTP `400`, error string | Validates required checkout field presence. |
| BB-51 | Checkout rejects COD above limit | Add high-value cart then checkout `COD` | HTTP `400`, contains `COD not allowed` | Enforces payment-method business constraint. |
| BB-52 | Checkout invalid payment method variant 1 | `POST /checkout` with `payment_method="UPI"` | HTTP `400`, error string | Validates unsupported payment enum value. |
| BB-53 | Checkout invalid payment method variant 2 | `POST /checkout` with `payment_method=123` | HTTP `400`, error string | Validates checkout input typing. |

| Test Case ID | API / Scenario | Input | Expected Output | Justification for Inclusion |
|---|---|---|---|---|
| BB-54 | Checkout invalid payment method variant 3 | `POST /checkout` with `payment_method=null` | HTTP `400`, error string | Validates null input rejection for payment method. |
| BB-55 | Checkout GST application correctness | Add item then checkout `CARD` | HTTP `200`; `gst_amount=5%` and `total=base+gst` | Verifies tax computation rules from API doc. |
| BB-56 | Wallet add zero amount rejection | `POST /wallet/add` with `amount=0` | HTTP `400`, `Invalid amount` | Validates lower bound for wallet top-up. |
| BB-57 | Wallet add max-boundary rejection | `POST /wallet/add` with `amount=100001` | HTTP `400`, error string | Verifies upper bound for wallet top-up. |
| BB-58 | Wallet add missing amount field | `POST /wallet/add` with `{}` | HTTP `400`, error string | Ensures required amount field enforcement. |
| BB-59 | Wallet pay invalid amount variant 1 | `POST /wallet/pay` with `amount=0` | HTTP `400`, error string | Validates minimum payable amount constraint. |
| BB-60 | Wallet pay invalid amount variant 2 | `POST /wallet/pay` with `amount=-1` | HTTP `400`, error string | Validates rejection of negative debits. |
| BB-61 | Wallet pay invalid amount variant 3 | `POST /wallet/pay` with very large amount | HTTP `400`, error string | Ensures insufficient-funds/invalid-amount handling. |
| BB-62 | Wallet pay exact deduction behavior | Add `10`, then pay `10` | Expected final balance returns to initial | Verifies precise debit accounting behavior. |
| BB-63 | Reviews average uses decimal precision | `GET /products/1/reviews` | HTTP `200`; `average_rating` equals arithmetic mean | Validates aggregate computation precision. |
| BB-64 | Reviews average zero when no reviews | `GET /products/14/reviews` | HTTP `200`; `reviews=[]`, `average_rating=0` | Verifies no-data aggregate behavior. |
| BB-65 | Add review invalid payload variant 1 | `rating=0`, valid comment | HTTP `400`, error string | Enforces rating lower bound. |
| BB-66 | Add review invalid payload variant 2 | `rating=6`, valid comment | HTTP `400`, error string | Enforces rating upper bound. |
| BB-67 | Add review invalid payload variant 3 | `rating="5"`, valid comment | HTTP `400`, error string | Enforces rating data type. |

| Test Case ID | API / Scenario | Input | Expected Output | Justification for Inclusion |
|---|---|---|---|---|
| BB-68 | Add review invalid payload variant 4 | `rating=5`, empty comment | HTTP `400`, error string | Enforces non-empty comment constraint. |
| BB-69 | Add review invalid payload variant 5 | `rating=5`, 201-char comment | HTTP `400`, error string | Enforces comment max length boundary. |
| BB-70 | Add review invalid payload variant 6 | Missing `comment` field | HTTP `400`, error string | Verifies required comment presence. |
| BB-71 | Order cancellation restores stock | Place order then cancel | Expected cancelled status and stock restored | Validates inventory compensation on cancellation. |
| BB-72 | Order cancellation missing order | `POST /orders/999999/cancel` | HTTP `404`, error string | Verifies missing-order handling. |
| BB-73 | Support ticket backward transition rejected | OPEN → IN_PROGRESS → OPEN | Expected HTTP `400` on backward transition | Enforces support ticket state-machine constraints. |
| BB-74 | Support ticket creation invalid variant 1 | Short subject payload | HTTP `400`, error string | Validates subject minimum length. |
| BB-75 | Support ticket creation invalid variant 2 | Empty message payload | HTTP `400`, error string | Validates message non-empty rule. |
| BB-76 | Support ticket creation invalid variant 3 | Empty payload `{}` | HTTP `400`, error string | Ensures required fields for ticket creation. |
| BB-77 | Support ticket update invalid status value | Update ticket with `status="REOPENED"` | HTTP `400`, error string | Validates strict support-ticket status enum. |
| BB-78 | Cart update subtotal/total consistency | Add item then `POST /cart/update` quantity `3` | Expected subtotal and total equal `unit_price × 3` | Verifies cart update arithmetic behavior from API doc. |
| BB-79 | Cart duplicate add subtotal consistency | Add same product twice (1 then 2) | Expected quantity `3`, subtotal `unit_price × 3` | Validates quantity merge and subtotal math. |
| BB-80 | Cart total equals sum of item subtotals | Add two products, fetch cart | Expected `cart.total = Σ item.subtotal` | Verifies multi-item total aggregation correctness. |
| BB-81 | Invoice GST and subtotal rule correctness | Checkout then fetch invoice | Expected subtotal, GST (5%), and total consistency | Validates invoice math and tax consistency. |

| Test Case ID | API / Scenario | Input | Expected Output | Justification for Inclusion |
|---|---|---|---|---|
| BB-82 | Checkout multi-qty GST total rule | Checkout quantity `2` for order product | Expected `total = subtotal + gst` with single GST application | Detects duplicate/missing tax application issues. |
| BB-83 | Loyalty redeem invalid payload variant 1 | `POST /loyalty/redeem` with `{}` | HTTP `400`, error string | Verifies required points field enforcement. |
| BB-84 | Loyalty redeem invalid payload variant 2 | `POST /loyalty/redeem` with `points=0` | HTTP `400`, error string | Validates minimum redeemable points rule. |
| BB-85 | Loyalty redeem invalid payload variant 3 | `POST /loyalty/redeem` with `points=-1` | HTTP `400`, error string | Validates negative points rejection. |
| BB-86 | Loyalty redeem invalid payload variant 4 | `POST /loyalty/redeem` with `points="1"` | HTTP `400`, error string | Validates points type enforcement. |
| BB-87 | Loyalty redeem insufficient points | `POST /loyalty/redeem` with very high points | HTTP `400`, error string | Verifies insufficient-points handling. |
| BB-88 | Checkout payment status mapping variant 1 | Checkout with `CARD` | HTTP `200`, `payment_status="PAID"` | Validates payment-status mapping from API doc. |
| BB-89 | Checkout payment status mapping variant 2 | Checkout with `COD` | HTTP `200`, `payment_status="PENDING"` | Validates payment-status mapping from API doc. |
| BB-90 | Checkout payment status mapping variant 3 | Checkout with `WALLET` | HTTP `200`, `payment_status="PENDING"` | Validates payment-status mapping from API doc. |
| BB-91 | Delete non-existent address | `DELETE /addresses/999999` | HTTP `404`, error string | Verifies missing-resource semantics for delete. |
| BB-92 | Update non-existent support ticket | `PUT /support/tickets/999999` | HTTP `404`, error string | Verifies missing-resource semantics for ticket update. |
| BB-93 | Closed support ticket cannot move to IN_PROGRESS | OPEN → IN_PROGRESS → CLOSED → IN_PROGRESS | Expected HTTP `400`, error string | Validates terminal-state enforcement in ticket workflow. |
| BB-94 | Wallet exact small deduction behavior | Add `1`, then pay `1` | Expected balance returns exactly to initial | Detects hidden fees/rounding drift on small wallet debits. |

## Why These Cases Matter for API Validation

1. **Header and identity validation coverage** (BB-01 to BB-07) confirms the API fails safely for malformed or missing identity context.
2. **API-doc validation on successful flow** (BB-08) ensures clients can rely on response structure and types.
3. **Input boundary validation for profile updates** (BB-09 to BB-11) verifies business rules at API layer, not just UI.
4. **Error payload consistency** across negative tests ensures predictable client-side error handling (`{"error": "..."}`).
5. **Catalog behavior and consistency checks** (BB-17 to BB-19) validate business-critical storefront correctness (visibility rules, discovery behavior, and exact pricing).
6. **Address validation and update coverage** (BB-20 to BB-24) checks schema validation, type safety, and post-update response correctness for address APIs.
7. **Cart and coupon rule validation** (BB-25 to BB-34) verifies quantity validation, payload schema enforcement, pricing arithmetic, and promotional constraint handling.
8. **Checkout, wallet, reviews, orders, and support coverage** (BB-47 to BB-76) validates pricing caps, payment and tax behavior, wallet accounting, review aggregates, order lifecycle integrity, and support ticket workflow rules.
9. **Advanced consistency and state-machine checks** (BB-77 to BB-94) extend coverage for update arithmetic, loyalty validation, payment-status mapping, non-existent resource handling, and terminal workflow transitions.

## Bug Report

### BR-01: Public product price is inconsistent with admin product price

- **Endpoint tested:** `GET /api/v1/products/{product_id}` (with cross-check against `GET /api/v1/admin/products`)
- **Request payload:**
  - **Method:** `GET`
  - **URL:** `http://localhost:8080/api/v1/products/123`
  - **Headers:**
    - `X-Roll-Number: 2024101018`
    - `X-User-ID: 1`
  - **Body:** none
- **Expected result (based on API doc):**
  - HTTP `200`
  - The `price` for product `123` in public product detail should exactly match admin source data (`225.05`).
- **Actual result observed:**
  - HTTP `200`

    - Public API returned `price = 230`, while admin API returned `price = 225.05`.
    - Assertion failure: `assert 230 == 225.05`.

## BR-02: Address update endpoint returns stale street value in response

- **Endpoint tested:** `PUT /api/v1/addresses/{address_id}` (with setup via `POST /api/v1/addresses`)
- **Request payload:**
    - **Method:** `PUT`
    - **URL:** `http://localhost:8080/api/v1/addresses/{newly_created_address_id}`
    - **Headers:**
        - `X-Roll-Number: 2024101018`
        - `X-User-ID: 1`
    - **Body:**
        - `{ "street": "QC Update Street <timestamp>", "is_default": true }`
- **Expected result (based on API doc):**
    - HTTP `200`
    - Response `address.street` should match the new street sent in request.
    - Response `address.is_default` should be `true`.
- **Actual result observed:**
    - HTTP `200`
    - `address.street` stayed as old value (`QC Seed Street 101`) instead of updated value (`QC Update Street <timestamp>`).
    - Assertion failure observed in test: `assert 'QC Seed Street 101' == 'QC Update Street ...'`.

## BR-03: Cart accepts zero quantity instead of rejecting request

- **Endpoint tested:** `POST /api/v1/cart/add`
- **Request payload:**
    - **Method:** `POST`
    - **URL:** `http://localhost:8080/api/v1/cart/add`
    - **Headers:**
        - `X-Roll-Number: 2024101018`
        - `X-User-ID: 1`
    - **Body:** `{ "product_id": 1, "quantity": 0 }`
- **Expected result (based on API doc/validation rules):**

    - HTTP `400`
    - Error payload indicating invalid quantity.
- **Actual result observed:**
    - HTTP `200`
    - Request accepted though quantity is zero.

## BR-04: Cart accepts payload missing required `quantity`

- **Endpoint tested:** `POST /api/v1/cart/add`
- **Request payload:**
    - **Method:** `POST`
    - **URL:** `http://localhost:8080/api/v1/cart/add`
    - **Headers:**
        - `X-Roll-Number: 2024101018`
        - `X-User-ID: 1`
    - **Body:** `{ "product_id": 1 }`
- **Expected result (based on API doc/validation rules):**
    - HTTP `400` (or `404` depending on validation order in implementation)
    - Error payload for missing required field.
- **Actual result observed:**
    - HTTP `200`
    - Item was accepted despite missing `quantity`.

## BR-05: Cart subtotal/total calculation is incorrect

- **Endpoint tested:** `GET /api/v1/cart` (after cart setup via `POST /api/v1/cart/add`)
- **Request payload:**
    - **Method:** `GET`
    - **URL:** `http://localhost:8080/api/v1/cart`
    - **Headers:**
        - `X-Roll-Number: 2024101018`
        - `X-User-ID: 1`
    - **Body:** none
- **Expected result (based on API doc financial rules):**
    - For product quantity `2`, item `subtotal = product_price × 2`
    - Cart `total` should match computed subtotal for single-item cart.
- **Actual result observed:**
    - Observed item `subtotal = -16`
    - Expected subtotal from product price was `240`
    - Assertion failure: `assert -16 == 240`.

### BR-06: FIRSTORDER coupon ignores configured max discount cap

- **Endpoint tested:** `POST /api/v1/coupon/apply`
- **Request payload:**
  - **Method:** `POST`
  - **URL:** `http://localhost:8080/api/v1/coupon/apply`
  - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 10`
  - **Body:** `{ "coupon_code": "FIRSTORDER" }` after adding product `1` with quantity `50`
- **Expected result:** discount should be capped at `150` (`min(total×0.15,150)`).
- **Actual result observed:** discount returned as `900`.

### BR-07: PERCENT20 coupon ignores configured max discount cap

- **Endpoint tested:** `POST /api/v1/coupon/apply`
- **Request payload:**
  - **Method:** `POST`
  - **URL:** `http://localhost:8080/api/v1/coupon/apply`
  - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 10`
  - **Body:** `{ "coupon_code": "PERCENT20" }` after adding product `1` with quantity `100`
- **Expected result:** discount should be capped at `200` (`min(total×0.20,200)`).
- **Actual result observed:** discount returned as `2400`.

### BR-08: Checkout GST test flow fails due to cart add rejection for checkout product

- **Endpoint tested:** `POST /api/v1/cart/add` (prerequisite for checkout GST validation)
- **Request payload:**
  - **Method:** `POST`
  - **URL:** `http://localhost:8080/api/v1/cart/add`
  - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 10`
  - **Body:** `{ "product_id": 67, "quantity": 1 }`
- **Expected result:** HTTP `200`, item added so checkout GST assertions can run.
- **Actual result observed:** HTTP `400`; downstream GST verification blocked.

### BR-09: Wallet pay does not deduct exact requested amount

- **Endpoint tested:** `POST /api/v1/wallet/pay`
- **Request payload:**
  - **Method:** `POST`

    - **URL:** `http://localhost:8080/api/v1/wallet/pay`
    - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 10`
    - **Body:** `{ "amount": 10 }`
- **Expected result:** wallet balance after add-10 then pay-10 should return to initial value.
- **Actual result observed:** balance after pay was lower than initial by `0.8` (non-exact deduction).

## BR-10: Reviews average rating loses decimal precision

- **Endpoint tested:** `GET /api/v1/products/1/reviews`
- **Request payload:**
    - **Method:** `GET`
    - **URL:** `http://localhost:8080/api/v1/products/1/reviews`
    - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 1`
    - **Body:** none
- **Expected result:** `average_rating` equals precise arithmetic mean of ratings (e.g., `3.2`).
- **Actual result observed:** API returned `3` (rounded/truncated), causing precision mismatch.

## BR-11: Order cancellation stock test blocked by add-to-cart rejection

- **Endpoint tested:** `POST /api/v1/cart/add` (prerequisite for place/cancel order flow)
- **Request payload:**
    - **Method:** `POST`
    - **URL:** `http://localhost:8080/api/v1/cart/add`
    - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 10`
    - **Body:** `{ "product_id": 67, "quantity": 1 }`
- **Expected result:** HTTP `200`, item added so order can be placed and cancellation stock restoration validated.
- **Actual result observed:** HTTP `400`, preventing full cancellation flow execution.

## BR-12: Support ticket workflow allows invalid backward status transition

- **Endpoint tested:** `PUT /api/v1/support/tickets/{ticket_id}`
- **Request payload:**
    - **Method:** `PUT`
    - **URL:** `http://localhost:8080/api/v1/support/tickets/{ticket_id}`
    - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 10`
    - **Body:** `{ "status": "OPEN" }` after moving ticket to `IN_PROGRESS`
- **Expected result:** HTTP `400`, backward transition should be rejected.

- **Actual result observed:** HTTP `200`, backward transition accepted.

## BR-13: Support ticket update accepts unsupported status value

- **Endpoint tested:** `PUT /api/v1/support/tickets/{ticket_id}`
- **Request payload:**
  - **Method:** `PUT`
  - **URL:** `http://localhost:8080/api/v1/support/tickets/{ticket_id}`
  - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 10`
  - **Body:** `{ "status": "REOPENED" }`
- **Expected result:** HTTP `400`, invalid status should be rejected.
- **Actual result observed:** HTTP `200` with success payload containing `status: REOPENED`.

## BR-14: Cart update arithmetic returns incorrect subtotal/total

- **Endpoint tested:** `POST /api/v1/cart/update` + `GET /api/v1/cart`
- **Request payload:**
  - **Method:** `POST`
  - **URL:** `http://localhost:8080/api/v1/cart/update`
  - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 1`
  - **Body:** `{ "product_id": 1, "quantity": 3 }`
- **Expected result:** item `subtotal = unit_price × 3` and cart total consistent.
- **Actual result observed:** subtotal returned `104` while expected was `360`.

## BR-15: Cart duplicate-add math is inconsistent

- **Endpoint tested:** `POST /api/v1/cart/add` + `GET /api/v1/cart`
- **Request payload:**
  - **Method:** `POST` (twice)
  - **URL:** `http://localhost:8080/api/v1/cart/add`
  - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 1`
  - **Body:** first `{ "product_id": 1, "quantity": 1 }`, second `{ "product_id": 1, "quantity": 2 }`
- **Expected result:** merged quantity `3`, subtotal `unit_price × 3`.
- **Actual result observed:** quantity became `3` but subtotal returned `104` instead of `360`.

## BR-16: Closed support ticket can be reopened to IN_PROGRESS

- **Endpoint tested:** `PUT /api/v1/support/tickets/{ticket_id}`
- **Request payload:**

    - **Method:** `PUT`
    - **URL:** `http://localhost:8080/api/v1/support/tickets/{ticket_id}`
    - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 10`
    - **Body:** `{ "status": "IN_PROGRESS" }` after ticket is already `CLOSED`
- **Expected result:** HTTP `400`, closed state should be terminal for this transition.
- **Actual result observed:** HTTP `200` with success payload and updated status.

## BR-17: Wallet pay applies hidden deduction/fee for exact payments

- **Endpoint tested:** `POST /api/v1/wallet/pay`
- **Request payload:**
    - **Method:** `POST`
    - **URL:** `http://localhost:8080/api/v1/wallet/pay`
    - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 10`
    - **Body:** `{ "amount": 1 }` (also reproduced with `{ "amount": 10 }`)
- **Expected result:** wallet balance after add-X then pay-X should return exactly to initial balance.
- **Actual result observed:** post-payment balance is lower than initial (extra deduction persists).

## BR-18: Cart total endpoint is inconsistent when multiple items are added

- **Endpoint tested:** `GET /api/v1/cart` (setup via `POST /api/v1/cart/add`)
- **Request payload:**
    - **Method:** `GET`
    - **URL:** `http://localhost:8080/api/v1/cart`
    - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 1`
    - **Body:** none
- **Expected result:** `cart.total` should equal the sum of all `item.subtotal` values for all cart items.
- **Actual result observed:** setup failed because adding product `67` returned HTTP `400`, so total-consistency flow could not execute as expected.

## BR-19: Invoice generation flow blocked by add-to-cart rejection for product 67

- **Endpoint tested:** `POST /api/v1/cart/add` (prerequisite for `GET /orders/{id}/invoice`)
- **Request payload:**
    - **Method:** `POST`
    - **URL:** `http://localhost:8080/api/v1/cart/add`

    - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 10`
    - **Body:** `{ "product_id": 67, "quantity": 1 }`
- **Expected result:** HTTP `200`, allowing checkout and invoice assertions (`subtotal`, `gst_amount`, `total_amount`) to run.
- **Actual result observed:** HTTP `400`, blocking invoice validation flow.

### BR-20: Multi-quantity checkout GST verification blocked by add-to-cart rejection

- **Endpoint tested:** `POST /api/v1/cart/add` (prerequisite for `POST /checkout`)
- **Request payload:**
    - **Method:** `POST`
    - **URL:** `http://localhost:8080/api/v1/cart/add`
    - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 10`
    - **Body:** `{ "product_id": 67, "quantity": 2 }`
- **Expected result:** HTTP `200`, then checkout should satisfy `total = subtotal + gst` for quantity 2.
- **Actual result observed:** HTTP `400`, checkout GST-total validation could not proceed.

### BR-21: Payment-status mapping for CARD cannot be validated due to cart add failure

- **Endpoint tested:** `POST /api/v1/cart/add` + `POST /api/v1/checkout`
- **Request payload:**
    - **Method:** `POST`
    - **URL:** `http://localhost:8080/api/v1/cart/add`
    - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 10`
    - **Body:** `{ "product_id": 67, "quantity": 1 }`
- **Expected result:** add-to-cart HTTP `200`, then checkout should return `payment_status = "PAID"` for `payment_method = "CARD"`.
- **Actual result observed:** add-to-cart returned HTTP `400`, so CARD payment-status expectation from API doc was not met in this flow.

### BR-22: Payment-status mapping for COD cannot be validated due to cart add failure

- **Endpoint tested:** `POST /api/v1/cart/add` + `POST /api/v1/checkout`
- **Request payload:**
    - **Method:** `POST`
    - **URL:** `http://localhost:8080/api/v1/cart/add`
    - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 10`

    - **Body:** `{ "product_id": 67, "quantity": 1 }`
- **Expected result:** add-to-cart HTTP `200`, then checkout should return `payment_status = "PENDING"` for `payment_method = "COD"`.
- **Actual result observed:** add-to-cart returned HTTP `400`, so COD payment-status expectation from API doc was not met in this flow.

### BR-23: Payment-status mapping for WALLET cannot be validated due to cart add failure

- **Endpoint tested:** `POST /api/v1/cart/add` + `POST /api/v1/checkout`
- **Request payload:**
    - **Method:** `POST`
    - **URL:** `http://localhost:8080/api/v1/cart/add`
    - **Headers:** `X-Roll-Number: 2024101018`, `X-User-ID: 10`
    - **Body:** `{ "product_id": 67, "quantity": 1 }`
- **Expected result:** add-to-cart HTTP `200`, then checkout should return `payment_status = "PENDING"` for `payment_method = "WALLET"`.
- **Actual result observed:** add-to-cart returned HTTP `400`, so WALLET payment-status expectation from API doc was not met in this flow.